Scoring Engine Build Pack

Scoring Engine Build Pack

Fantasy Cricket Match Day Optimizer · Phase 3, built on its own

This pack contains everything Claude Code needs to build **only the scoring engine**. Part A is the prompt to paste. Part B is the full specification it must follow.

Part A: The prompt

Paste this into Claude Code, with this file saved in the repo as `docs/SCORING_ENGINE_BUILD.md`.

> You are building **only the scoring engine** for my project "Fantasy Cricket Match Day Optimizer". The full specification is in `docs/SCORING_ENGINE_BUILD.md` (Part B). Read all of Part B before doing anything. Also read `docs/RUBRIC.md` and `docs/NEON_SCHEMA.md` if they are in the repo; where they disagree with Part B, Part B wins for this task.
>
> **Scope:** the scoring engine, the minimum data plumbing it needs, and a command-line tool to run it on a real match. **Do not build** Gemini, the optimizer, the validity gate, the learning loop, or any UI.
>
> **Before writing any code:** 1. Inspect the repository and tell me what already exists, especially `engine/points.py`, `engine/factors.py`, the backtest code, the `config/` files, and the Neon tables created by backtest run 1. Run the existing tests and report the result. 2. Make one real call to each CricketData.org endpoint you plan to use, save the raw responses in `tests/fixtures/cricketdata/`, and show me the field names you will rely on. Do not write field names from memory. 3. Show me a plan: files to create or change, functions with signatures, migrations, and tests. Wait for my go-ahead.
>
> **Rules you must follow:** - Never invent facts, values, API fields or library functions. If something is missing, write `TODO-CONFIRM` and ask me. - Never break working code. Commit before and after each change, run the tests before and after, keep diffs small, and stop if a passing test fails. - Database changes are additive migrations only. Copy data when renaming tables; never drop anything without my yes. - Never change existing behaviour of `engine/points.py` or `engine/factors.py`. You may add new functions to them.
>
> Build in the order of Part B, section 15. After each step, show me what you built and the tests that pass.

Part B: Specification

1. What the scoring engine is

The scoring engine is a **calculator**. For one match and a given list of players, it predicts each player's **expected Dream11 points** (the "base score") by applying the **rubric**: 8 factors combined by a weighted average.

- It is **code only**. No AI, no randomness. The same inputs always give the same outputs.
- It does **not** choose players, captains or teams. It does **not** read news. Those are later steps (Gemini moderator, optimizer).
- Its output is saved to Neon and later passed to Gemini and the optimizer.

Analogy: the rubric is the marking scheme; the scoring engine is the examiner who applies it mechanically to every player.

2. Scope

In scope:

Item	File
Settings loader	`engine/settings.py`
Active rubric loader (with borrowing for unreliable groups)	`engine/rubric.py`
Player name matching across sources	`engine/matching.py`
Provisional points from CricketData.org scorecards	`engine/fresh.py`
Fallback chain for players with little history	`engine/fallback.py`
The scorer	`engine/score.py`
CricketData.org client with cache and daily budget	`engine/sources/cricketdata.py`
Migrations for the tables in section 7	`db/migrations/`
Command-line runner	`engine/cli.py`
Tests	`tests/engine/`

Out of scope (do not build): Gemini (any job), optimizer, validity gate, bench, learning loop, results entry, any UI, cron jobs, deployment.

3. What already exists (from backtest run 1)

Run 1 built and used:

- `engine/points.py`: Dream11 points from Cricsheet ball-by-ball data.
- `engine/factors.py`: the 8 factors and the thin-data rule.
- A Cricsheet downloader and parser in `/backtest`.
- Neon tables including `bt_player_match_points`, `bt_predictions` and `rubric_versions`.
- `config/dream11_points.yaml` with **provisional** values (not yet copied from the Dream11 app).
- `tests/test_no_peeking.py`.

Reuse these. Do not rebuild or rewrite them. If something in them doesn't fit this spec, report it with evidence and propose the smallest change.

4. Rules

4.1 No hallucination

1. Every external fact comes from something you actually checked: a real API response saved as a fixture, official documentation, the repo, or this spec.
2. API field names come from saved real responses, never from memory.
3. Dream11 values come only from `config/`. Missing value → stop and ask.
4. Use only library functions you have confirmed exist in the installed version.
5. Numbers in your summaries are copied from real outputs.
6. Unverified items are marked `TODO-CONFIRM` in code and listed in your summary.

4.2 Don’t break working code

1. Commit before and after each step; one branch for this phase.
2. Run all existing tests before starting and after every step. If a passing test fails, stop and report.
3. Small diffs; no unrequested refactors or renames.
4. `points.py` and `factors.py`: add functions only; don’t change existing behaviour. Rerun `test_no_peeking.py` after touching them.
5. Migrations are additive and tested on a Neon development branch first.
6. Never overwrite `config/` values.

5. Dependencies and how to handle them

Dependency	State now	What the engine does
Rubric weights	Run 1 weights exist but are **provisional** (points values not from the app)	Use the active `rubric_versions` row. If its `points_values_final` is false or missing, mark every output `provisional_weights = true` and log it. Never block
`format_scaling` (fallback step 2)	Not produced yet (comes from backtest run 2)	If the table is missing or has no row for the needed pair, skip step 2 and go to step 3. Record `fallback_note = "format scaling not available"`
`debutant_averages` (fallback step 4)	Not produced yet	If missing, return `base_score = null`, `fallback_step = 4`, `fallback_note = "no data"`, and flag thin data. Do **not** substitute any other number (for example a median of other players): that would be invented. Report how many players this affects
Dream11 points values	Provisional	Use them; mark provisional fill rows accordingly. Values will be replaced later by us
Credits and Dream11 roles	Entered by Aryan later via the UI	Not needed by the engine. The engine scores whatever player list it is given

6. The rubric the engine applies

6.1 Format groups

`format_group` = format × gender: `T20_men`, `T20_women`, `ODI_men`, `ODI_women`, `Test_men`, `Test_women`.

Cricsheet mapping (from run 1): IT20 and T20 → T20 (franchise and domestic included); ODI and ODM → ODI; Test and MDM → Test. The Hundred (5-ball overs) is not supported: if a match is a Hundred match, return an error “Format not supported”.

6.2 The 8 factors

Each factor = the player’s average Dream11 points in situation X, using only matches **in the same format group** and **dated strictly before the match date** (same-date matches never see each other).

#	Factor	Definition
1	form	Last 10 matches (any competition); the most recent 5 count double: `(2 × sum(last 5) + sum(previous 5)) / (2 × n_recent + n_previous)`
2	venue	Average at this venue (venue names normalised by dropping a “, city” suffix)
3	opponent	Average against this opposing team
4	current_touch	His last 5 matches in **other** formats played in the last 30 days; for each: ratio = points ÷ his average in that format (only formats where his average is positive and backed by at least 3 matches). Value = form × mean(ratio). None qualify → value = form
5	opportunity	Average in matches where his batting band (1–3, 4–5, 6–7, 8+; batting position = order of first appearance at the crease, did not bat = 8+) **and** bowling band (did not bowl / under half the maximum quota / half or more; Tests: most overs in one innings, under 20 / 20+) both equal his usual bands (most frequent in his last 10)
6	career	Average over all earlier matches in the data window
7	bat_first_or_chase	Average when his team batted first vs chased, matching today’s situation. **Pre-toss: value = form.** Post-toss: use the toss result
8	pitch_type	Venue classed batting- or bowling-friendly by its average first-innings score in earlier matches vs the format average (needs at least 5 earlier matches at the venue, else

		unknown → value = form). Factor = his average at venues of the same type

Use the existing `engine/factors.py` for these. If it is missing anything in this table (for example the pre-toss neutral rule), add it as a new function or parameter without changing existing behaviour, and report it.

6.3 Thin-data rule

For venue, opponent, opportunity, bat_first_or_chase and pitch_type:

```
trust = min(n / 10, 1)                 n = matches behind that factor
value = trust × factor_average + (1 − trust) × form
n = 0  →  value = form, trust = 0
```

Form, career and current_touch have trust = 1 (form and career are averages over all available history; current_touch already falls back to form).

6.4 Combination

```
base_score = Σ weight_i × value_i          (weights from the active rubric version; Σ weights = 1)
```

6.5 Unreliable groups

If the active rubric row for the format group has `reliable = false` (or is missing), use the active weights of the same format's other gender (`Test_women` → `Test_men`). Record `borrowed_from_group` and set `thin_data = true` for every player in that match.

7. Database tables needed (subset of NEON_SCHEMA.md)

Create only what's missing, with additive migrations. Column names exactly as below.

Table	Columns
teams	id, name, short_code, gender, cricketdata_id, cricsheet_name, created_at
venues	id, name, city, country, lat, lon, cricketdata_id, cricsheet_name, created_at
players	id, full_name, gender, cricsheet_id, cricketdata_id, image_url, image_source, batting_style, bowling_style, updated_at, created_at
player_name_links	id, player_id, source (dream11 / cricketdata / cricsheet), source_name, confirmed_by_aryan, confirmed_at, created_at
matches	id, cricketdata_id, cricsheet_id, format, gender, format_group, series_name, venue_id, team_a_id, team_b_id, start_time, status, toss_winner_team_id, toss_decision, batting_first_team_id, rain_affected, xi_announced, updated_at, created_at
match_xi	match_id, team_id, player_id, source, fetched_at
player_match_points	id, player_id, match_id, format_group, batting_points, bowling_points, fielding_points, total_points, batting_position, overs_bowled, retired_hurt, source (cricsheet / scorecard), is_provisional, updated_at, created_at; unique (player_id, match_id)
api_cache	id, source, request_key, response (jsonb), fetched_at, expires_at
api_usage	source, date, calls; primary key (source, date)
settings	key, value (jsonb), updated_by, updated_at
rubric_versions	existing table from run 1; **add** any missing columns: validation_mae, benchmark_50_50_mae, reliable, borrowed_from_group, points_values_final, is_active
runs	id, match_id, run_type (pre_toss / post_toss), status, failed_step, error_message, rubric_version_id, data_stale, started_at, finished_at, created_at
predictions	id, run_id, player_id, factors (jsonb), trust (jsonb), fallback_step, fallback_note, thin_data, provisional_inputs, provisional_weights, borrowed_from_group, base_score, created_at

Run 1 tables: migrate `bt_player_match_points` → `player_match_points` with `source = 'cricsheet'`, `is_provisional = false`, **copying the data** and checking row counts match. Update code references in the same commit. Keep `bt_player_match_points` until I confirm. Leave `bt_predictions` alone in this phase.

Settings to seed (only if absent; never overwrite): `form_window` 10, `form_recent_double` 5, `current_touch_matches` 5, `current_touch_days` 30, `thin_data_full_trust` 10, `cold_start_min_matches` 3, `pitch_type_min_matches` 5, `thin_data_trust_threshold` 0.5, `cricketdata_daily_limit` 95.

8. Data sources for this phase

8.1 CricketData.org

Needed for: upcoming fixtures, squads, confirmed XI and toss, scorecards of recent matches not yet on Cricsheet, player info (career stats; image URL if present).

1. Read CricketData.org's official documentation. Make one real call per endpoint you need with `CRICKETDATA_API_KEY`. Save raw responses under `tests/fixtures/cricketdata/<endpoint>.json`. Write the client against those saved responses.
2. **Cache** every response in `api_cache` (key = endpoint + parameters). Suggested freshness: fixtures 6 hours, squads 6 hours, match info (XI, toss) 10 minutes, scorecards 7 days once a match is finished, player info 30 days.
3. **Budget:** count calls in `api_usage`. Stop live calls at `cricketdata_daily_limit` (95 of the 100 free calls). When stopped or the API fails, use the cache and set `runs.data_stale = true`.
4. Tests must use the saved fixtures; no live calls in tests.

8.2 Cricsheet

Reuse the run 1 downloader/parser. Add a command `engine.cli refresh-cricsheet` that downloads only new matches, computes points with `points.py`, and upserts `player_match_points` with `source = 'cricsheet'`, replacing any `scorecard` row for the same player and match.

9. Module specifications

9.1 `engine/settings.py`

`get(key) -> value` from `settings`, with typed accessors. Raise a clear error if a required key is missing.

9.2 `engine/rubric.py`

`active_weights(format_group) -> RubricChoice(weights: dict[str, float], version_id, provisional: bool, borrowed_from: str | None)`. - Picks the row with `is_active = true` for the group. - Applies section 6.5 borrowing. - `provisional = not points_values_final`. - Validates: 8 known factor keys, weights ≥ 0, sum within 0.001 of 1. Otherwise raise.

9.3 `engine/matching.py`

`resolve(source, source_name, team_hint=None) -> player_id | None`. 1. Exact link in `player_name_links` → return it. 2. Exact normalised full-name match (case, accents, punctuation removed) within the team's players → link and return. 3. Initial-plus-surname match ("V Kohli" ↔ "Virat Kohli") within the team, only if exactly one candidate → link and return. 4. Otherwise return None and add the name to an "unmatched" list in the result. (Aryan confirms later in the UI; out of scope now.) Never guess between two candidates.

9.4 `engine/fresh.py`

`fill_recent(player_ids, match_date)`: - For each player, find matches in the last 30 days (from CricketData.org match lists) that are finished but not yet in `player_match_points` from Cricsheet. - Fetch each scorecard once (cached), compute points with a **new** function `points.from_scorecard(scorecard_row, format)` added to `points.py`, using the same points table. - Scorecards don't have every ball-by-ball detail. Apply only point types the scorecard actually supports (as confirmed from the saved fixture); list unsupported point types in the function's docstring and in your summary. - Upsert with `source = 'scorecard'`, `is_provisional = true`. Never overwrite a `cricsheet` row.

9.5 `engine/fallback.py`

For a player with fewer than `cold_start_min_matches` (3) earlier matches in the format group:

Step	Condition	Value	Note
2	Has earlier matches in other formats, and `format_scaling` has a row for (other group → this group)	his average in that other group × ratio (if several, use the group with the most matches)	"other formats × scaling"
3	No usable Cricsheet history, but CricketData.org player info has career stats for this format	per-match estimate using only the point types the stats support: runs per innings × points per run, wickets per innings × points per wicket, catches per match × points per catch (values from the points table)	"career stats estimate"
4	Nothing usable	`debutant_averages` value for the group if available; else `base_score = null`	"debutant average" or "no data"

All steps 2 to 4 set `thin_data = true`. Steps only use fields confirmed in saved fixtures; if career stats don't contain a field, that part is skipped and noted.

9.6 `engine/score.py`

```
def score_match(match_id: str, player_ids: list[str], run_type: str,
                run_id: str | None = None) -> ScoreResult
```

Steps: 1. Load the match; check `format_group` is supported. 2. If `run_id` is None, create a `runs` row (`status = 'scoring'`). 3. `rubric.active_weights(format_group)`. 4. `fresh.fill_recent(player_ids, match_date)`. 5. For each player: a. Count earlier matches in the format group. If fewer than 3 → fallback chain (section 9.5); factors are not computed. b. Otherwise compute the 8 factor values and trusts with `factors.py` (section 6), applying the pre-toss rule for bat_first_or_chase. c. `base_score = Σ weight × value`. d. `thin_data = fallback_step ≥ 2` **or** `Σ (weight_i × trust_i) < thin_data_trust_threshold` (0.5) **or** the group borrowed weights. e. `provisional_inputs = true` if any history row used has `is_provisional = true`. 6. Save one `predictions` row per player; set `runs.status = 'done'` (or `failed` with `failed_step` and a plain-English `error_message`). 7. Return `ScoreResult`.

9.7 Output contract

```
@dataclass
class PlayerScore:
    player_id: str
    base_score: float | None
    factors: dict[str, float]          # 8 values (empty if fallback used)
    trust: dict[str, float]            # 8 trusts (empty if fallback used)
    weights: dict[str, float]          # weights applied
    fallback_step: int                 # 1 to 4
    fallback_note: str | None
    thin_data: bool
    provisional_inputs: bool

@dataclass
class ScoreResult:
    run_id: str
    match_id: str
    format_group: str
    run_type: str                      # pre_toss / post_toss
    rubric_version_id: str
    provisional_weights: bool
    borrowed_from_group: str | None
    data_stale: bool
    players: list[PlayerScore]
    unmatched_names: list[str]
    warnings: list[str]                # e.g. "format scaling not available"
```

9.8 `engine/cli.py`

```
python -m engine.cli upcoming                        # list next 7 days' matches (from CricketData.org)
python -m engine.cli score --match <id> --run-type pre_toss|post_toss
python -m engine.cli refresh-cricsheet
python -m engine.cli budget                          # today's CricketData.org calls
```

`score` builds the player list itself for testing: post-toss with XI announced → confirmed XI of both teams; otherwise both squads. It prints a table sorted by base score: name, team, base score, the 8 factor values, fallback step, thin-data flag. It also prints the rubric version used, whether weights are provisional, and any warnings.

10. Pre-toss vs post-toss

	Pre-toss	Post-toss
bat_first_or_chase	value = form (neutral), trust = 0	uses the actual toss result
Player list	given by the caller	given by the caller (normally the confirmed XI)
`runs.run_type`	`pre_toss`	`post_toss`

11. Worked example (made-up numbers, to show the maths only)

A T20 batter with 30 earlier T20 matches. Made-up weights: form 0.10, venue 0.15, opponent 0.05, career 0.70, others 0.

- form = 40 (trust 1); career = 35 (trust 1).
- venue: 2 matches averaging 90 → trust 0.2 → value = 0.2 × 90 + 0.8 × 40 = 50.
- opponent: 12 matches averaging 30 → trust 1 → value = 30.
- base_score = 0.10 × 40 + 0.15 × 50 + 0.05 × 30 + 0.70 × 35 = 4 + 7.5 + 1.5 + 24.5 = **37.5**.
- Σ weight × trust = 0.10 × 1 + 0.15 × 0.2 + 0.05 × 1 + 0.70 × 1 = 0.88 → not thin data.

Use this exact case as a unit test with these made-up numbers.

12. Error handling

Situation	Behaviour
CricketData.org limit or outage	Use cache; `data_stale = true`; warning “Using saved data from {time}”
No active rubric for group and no borrowable group	`runs.status = failed`, message “No rubric weights for {group}. Run the backtest.”
Rubric weights invalid	Fail with the reason
Unsupported format (e.g. The Hundred)	Fail: “Format not supported”
Player can’t be matched	Added to `unmatched_names`; not scored
Required config/settings value missing	Fail with “Missing setting {key}”
Any exception mid-run	`runs.status = failed`, `failed_step`, plain-English message; already-saved predictions are kept

13. Tests

All tests use saved fixtures; no live calls.

1. **No peeking:** factors for a match never use matches on or after its date (extend the existing test).
2. **Determinism:** scoring the same match twice gives identical results.
3. **Worked example** (section 11) gives 37.5 and not thin data.
4. **Thin-data rule:** n = 0, 5, 10, 15 give trust 0, 0.5, 1, 1.
5. **Pre-toss:** bat_first_or_chase value equals form.
6. **Fallback chain:** each step triggers under its condition; missing `format_scaling` skips to step 3; missing everything gives `base_score = null` with “no data”.
7. **Borrowing:** an unreliable group uses the other gender’s weights and sets thin data.
8. **Rubric validation:** weights not summing to 1 raise an error.
9. **Matching:** exact, initials, ambiguous (returns None), unknown.
10. **Provisional fill:** a scorecard row is created once; a later Cricsheet row replaces it, never duplicates.
11. **Budget:** at the limit, no live call is made and `data_stale` is set.
12. **Migration:** row count of `player_match_points` (source cricsheet) equals `bt_player_match_points`.

14. Acceptance criteria (phase done when)

- All existing tests and all new tests pass.
- `python -m engine.cli score --match <a real upcoming match>` prints base scores for every listed player, the rubric version, the provisional flag, and warnings.
- Running it twice gives identical output.
- `predictions` and `runs` rows exist for that run.
- Your summary lists: files changed, migrations applied, real fields used from CricketData.org (with fixture paths), point types unsupported by scorecards, players hitting each fallback step, and every `TODO-CONFIRM`.

15. Build order

1. Repo inspection + existing tests (report).
2. CricketData.org endpoint discovery: real calls, saved fixtures, field list (report).
3. Migrations (section 7) on a Neon development branch, including the `bt_player_match_points` copy (report row counts).
4. `settings.py`, `rubric.py` + tests.
5. `sources/cricketdata.py` with cache and budget + tests.
6. `matching.py` + tests.
7. `points.from_scorecard` and `fresh.py` + tests.
8. `fallback.py` + tests.
9. `score.py` + tests (including the worked example).
10. `cli.py`; run on one real upcoming match; show the output.

After each step: commit, run all tests, show results, wait for go-ahead.

16. Never do

- Build anything out of scope (Gemini, optimizer, UI, learning).
- Invent API fields, Dream11 values, player data or weights.
- Fill a missing number with a guess (including “median of other players”).
- Change existing behaviour of `points.py` or `factors.py`.
- Drop or overwrite tables, config or settings.
- Make live API calls in tests.

17. Open items to confirm during the build

- CricketData.org endpoint names and fields (fixtures, squads, XI, toss, scorecards, player info, image URL).
- Which Dream11 point types a CricketData.org scorecard supports.
- Whether the existing `factors.py` already implements the pre-toss neutral rule and the opportunity bands exactly as in section 6.2.